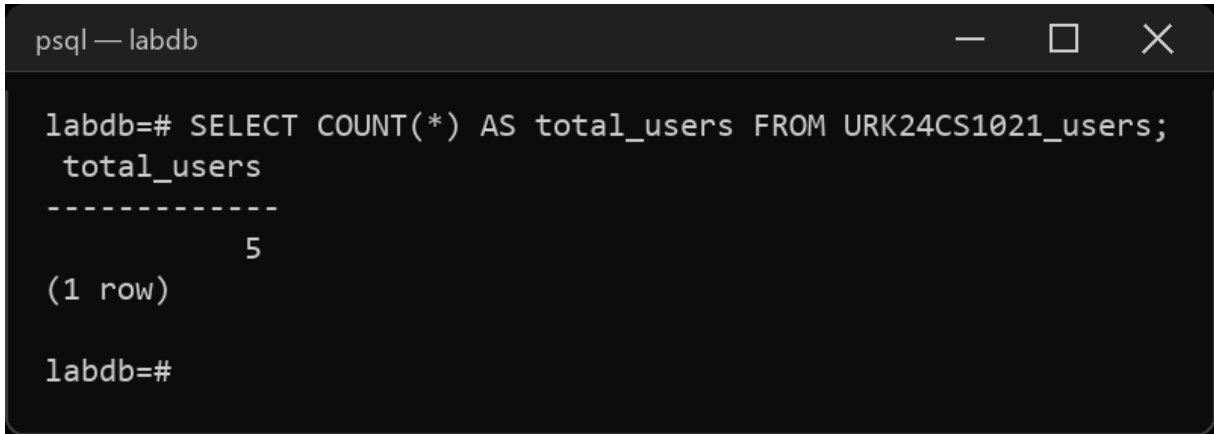


Ex. No. 3 Date: 05/08/2026	ADVANCED SQL	URK24CS1021
Aim To execute the given commands making use of aggregate functions, group by clause and order by clause. Description Aggregate functions take a collection of values and return a single value as the result: SUM returns the total, AVG the average, COUNT the number of tuples, and MIN and MAX the smallest and largest value in the collection. SUM and AVG need numeric input, while COUNT, MIN and MAX also work on non-numeric types. The DISTINCT keyword inside an aggregate expression eliminates duplicates before the function is applied. The GROUP BY clause applies an aggregate function to groups of tuples — tuples sharing the same value on every attribute named in the clause are placed in one group — and the ORDER BY clause returns the result of a query in sorted order, ASC for ascending and DESC for descending. Questions 1. List the number of users in the user table. <pre>SELECT COUNT(*) AS total_users FROM URK24CS1021_users;</pre>  <pre>psql — labdb labdb=# SELECT COUNT(*) AS total_users FROM URK24CS1021_users; total_users ----- 5 (1 row) labdb=#</pre> 2. List the number of events conducted during 2023. <pre>SELECT COUNT(*) AS events_2023 FROM URK24CS1021_event WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';</pre>		

```
psql — labdb

labdb=# SELECT COUNT(*) AS events_2023 FROM URK24CS1021_event
labdb(# WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';
events_2023
-----
                4
(1 row)

labdb=#
```

3. Calculate the total price for the confirmed tickets.

```
SELECT SUM(price) AS confirmed_total FROM URK24CS1021_ticket
WHERE status = 'confirmed';
```

```
psql — labdb

labdb=# SELECT SUM(price) AS confirmed_total FROM URK24CS1021_ticket
labdb(# WHERE status = 'confirmed';
confirmed_total
-----
            323.50
(1 row)

labdb=#
```

4. Find the minimum, maximum and average time taken for the given events.

```
SELECT MIN(endtime - eventtime) AS min_duration,
       MAX(endtime - eventtime) AS max_duration,
       AVG(endtime - eventtime) AS avg_duration
FROM URK24CS1021_event;
```

```
psql — labdb

labdb=# SELECT MIN(endtime - eventtime) AS min_duration,
labdb(#      MAX(endtime - eventtime) AS max_duration,
labdb(#      AVG(endtime - eventtime) AS avg_duration
labdb(# FROM URK24CS1021_event;
  min_duration | max_duration | avg_duration
-----+-----+-----
    03:30:00   | 12:00:00   | 08:00:00
(1 row)

labdb=#
```

5. Retrieve distinct number of users.

```
SELECT COUNT(DISTINCT userid) AS distinct_users FROM URK24CS1021_ticket;
```

```
psql — labdb

labdb=# SELECT COUNT(DISTINCT userid) AS distinct_users FROM URK24CS1021_ticket;
  distinct_users
-----
              5
(1 row)

labdb=#
```

6. List names of events was more than 10 hours.

```
SELECT name, endtime - eventtime AS duration FROM URK24CS1021_event
WHERE endtime - eventtime > INTERVAL '10 hours';
```

```
psql — labdb

labdb=# SELECT name, endtime - eventtime AS duration FROM URK24CS1021_event
labdb(# WHERE endtime - eventtime > INTERVAL '10 hours';
      name      | duration
-----+-----
Tech Conference | 12:00:00
Book Fair       | 12:00:00
(2 rows)

labdb=#
```

7. List the seat numbers of the users those whose tickets are confirmed.

```
SELECT userid, seatnumber FROM URK24CS1021_ticket
WHERE status = 'confirmed' ORDER BY seatnumber;
```

```
psql — labdb

labdb=# SELECT userid, seatnumber FROM URK24CS1021_ticket
labdb(# WHERE status = 'confirmed' ORDER BY seatnumber;
  userid | seatnumber
-----+-----
    101  | A12
    102  | B06
    104  | C21
    105  | C22
(4 rows)

labdb=#
```

8. List user name and phone numbers those whose tickets reserved.

```
SELECT u.name, u.phone FROM URK24CS1021_users u, URK24CS1021_ticket t
WHERE u.id = t.userid AND t.status = 'reserved';
```

```
psql — labdb

labdb=# SELECT u.name, u.phone FROM URK24CS1021_users u, URK24CS1021_ticket t
labdb(# WHERE u.id = t.userid AND t.status = 'reserved';
  name      | phone
-----+-----
Bob Smith   | 9000000002
Alice Johnson | 9000000001
Carol Lee   | 9000000003
(3 rows)

labdb=#
```

9. List the user details in the ascending order of their ticket price.

```
SELECT u.id, u.name, u.email, u.phone, t.price
FROM URK24CS1021_users u, URK24CS1021_ticket t
WHERE u.id = t.userid ORDER BY t.price ASC;
```

psql — labdb

```
labdb=# SELECT u.id, u.name, u.email, u.phone, t.price
labdb(# FROM URK24CS1021_users u, URK24CS1021_ticket t
labdb(# WHERE u.id = t.userid ORDER BY t.price ASC;
```

id	name	email	phone	price
105	Emma Watson	emma@mail.com	9000000005	25.00
102	Bob Smith	bob@mail.com	9000000002	28.50
103	Carol Lee	carol@mail.com	9000000003	30.00
101	Alice Johnson	alice@mail.com	9000000001	90.00
104	David Kim	david@mail.com	9000000004	120.00
102	Bob Smith	bob@mail.com	9000000002	150.00
101	Alice Johnson	alice@mail.com	9000000001	150.00
103	Carol Lee	carol@mail.com	9000000003	200.00

(8 rows)

labdb=#

10. Modify the ticket price of the user based on the following: (a) if the price is greater than 30, give a concession of 2%; (b) if the price is lesser than or equal to 30, give a concession of 4%.

```
SELECT ticketid, price FROM URK24CS1021_ticket ORDER BY ticketid;
UPDATE URK24CS1021_ticket SET price = CASE
    WHEN price > 30 THEN price * 0.98
    ELSE price * 0.96
END;
SELECT ticketid, price FROM URK24CS1021_ticket ORDER BY ticketid;
```

psql — labdb

```
labdb=# SELECT ticketid, price FROM URK24CS1021_ticket ORDER BY ticketid;
 ticketid | price
```

```
-----+-----
      1 | 150.00
      2 | 150.00
      3 | 200.00
      4 | 120.00
      5 |  25.00
      6 |  90.00
      7 |  28.50
      8 |  30.00
```

(8 rows)

```
labdb=# UPDATE URK24CS1021_ticket SET price = CASE
labdb(#      WHEN price > 30 THEN price * 0.98
labdb(#      ELSE price * 0.96
labdb(# END;
UPDATE 8
```

```
labdb=# SELECT ticketid, price FROM URK24CS1021_ticket ORDER BY ticketid;
 ticketid | price
```

```
-----+-----
      1 | 147.00
      2 | 147.00
      3 | 196.00
      4 | 117.60
      5 |  24.00
      6 |  88.20
      7 |  27.36
      8 |  28.80
```

(8 rows)

```
labdb=#
```

11. List the Venue name and address in each country.

```
SELECT country, name, address FROM URK24CS1021_location
ORDER BY country, name;
```

```
psql — labdb

labdb=# SELECT country, name, address FROM URK24CS1021_location
labdb=# ORDER BY country, name;
   country   |          name          |          address
-----+-----+-----
India        | Chennai Trade Centre   | Mount Poonamallee Rd
Singapore    | Marina Bay Sands       | 10 Bayfront Ave
UK           | Wembley Stadium        | Wembley
USA          | Madison Square Garden  | 4 Pennsylvania Plaza
(4 rows)

labdb=#
```

12. List all the city name in Alphabetical order.

```
SELECT city FROM URK24CS1021_location ORDER BY city ASC;
```

```
psql — labdb

labdb=# SELECT city FROM URK24CS1021_location ORDER BY city ASC;
   city
-----
Chennai
London
New York
Singapore
(4 rows)

labdb=#
```

Result

The queries were executed and the desired output was obtained.